

From an Untriggerable XSS to a Dream Chain of XSS in Multiple Critical APIs

By h0af3ng · Translated from the Chinese original

Not long ago I joined a crowd-testing engagement that required hunting bugs on a vendor's systems.

While testing one system, I found a very interesting interface whose output points could be manipulated from two other sites, ultimately leading to XSS triggers in multiple places.

0x01: First Look at the Vulnerabilities

The system's URL looked like this: `https://xxx.com/passport/?fromUrl=https://xxx.com/`

A URL like this is prone to open redirect, XSS, and CRLF injection. There are also parameters like these:

```
redirect ref redirect_to redirect_url url jump jump_to
target to link linkto domain server
```

I did some quick-and-dirty testing:

```
https://xxx.com/passport/?fromUrl=https://www.baidu.com
https://xxx.com/passport/?fromUrl=https://www.baidu.com.eval.com
https://xxx.com/passport/?fromUrl=https://www.baidu.com@www.eval.com
https://xxx.com/passport/?fromUrl=javascript:alert(1)
https://xxx.com/passport/?fromUrl=xxxx%0D%0ASet-Cookie:hacker=crlf
```

(To check whether the CRLF injection works, just look for the string `hacker=crlf` in the cookies.)

Simple testing showed none of the above fired.

So the only option was to dig into the system's functional points and interfaces.

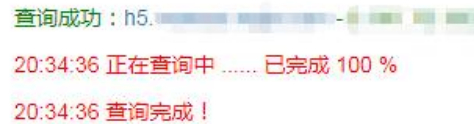
0x02: Deep into the Tiger's Den



Let's log in to the system and dig deeper.

(PS: after login the system asked me to fill in some profile info; my personal habit in that situation is to register with "><img/src=1> as the input.)

Thanks to earlier reconnaissance, subdomain brute-forcing successfully enumerated a mobile site: h5.xxx.com



```
查询成功 : h5.
20:34:36 正在查询中 ..... 已完成 100 %
20:34:36 查询完成 !
```

Since we're already logged in on the xxx.com domain, visiting

`https://h5.xxx.com/`

keeps us logged in as well (common with big vendors' account systems and SSO single sign-on).



Click "Mine" in the top-right corner to enter the personal center page.



Here we jump into the personal center.

Spotting the "personal profile" feature means there may be:

- XSS via profile modification
- IDOR: modifying other users' profile information
- Deleting other users' information

- Swapping some parameter value to view other people's profiles... and a whole series of logic issues

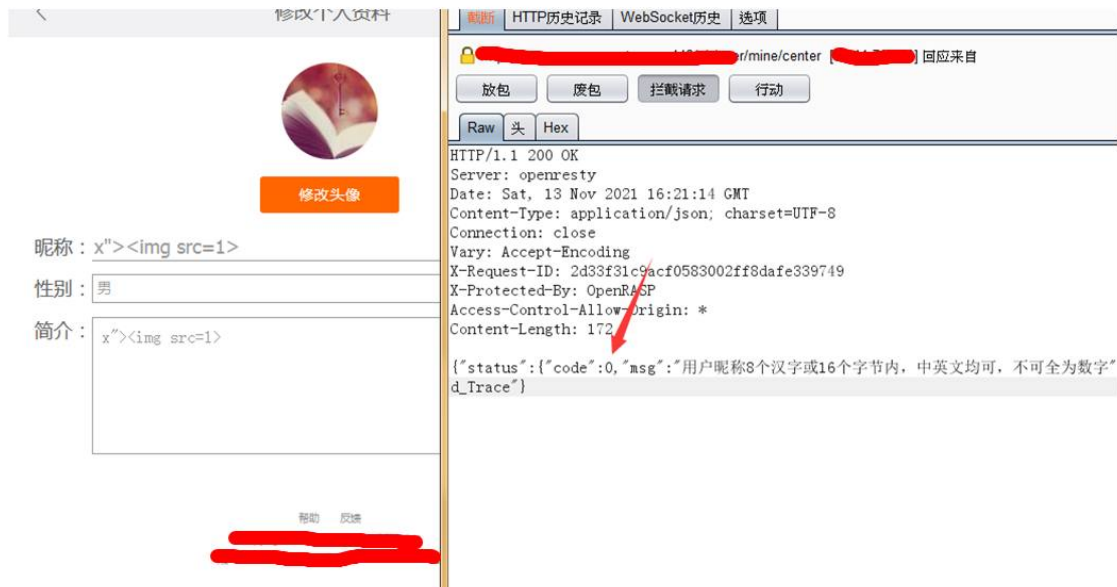
I edited my profile and embedded XSS into the page context:

```
x"><svg/onload=alert(1)>
```

(I forgot to switch the payload back when taking the screenshot — make do with it :))

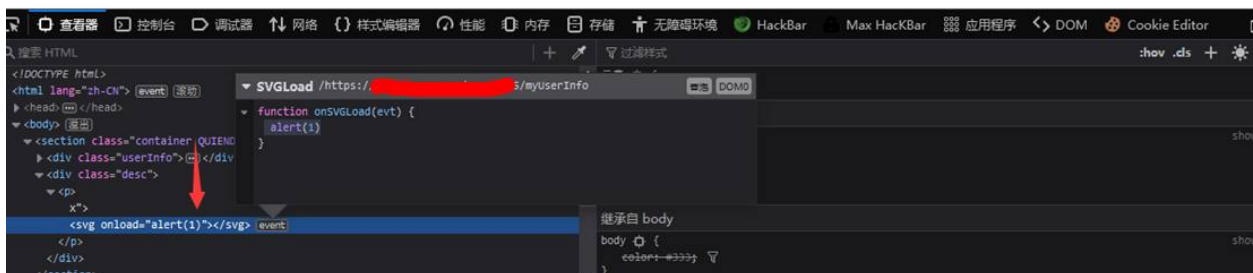


On saving, I found the username had a length restriction. I modified the response packet in Burp Suite, but to no avail.



So I had to fill the form in again, this time putting the payload only in the "bio" field with an arbitrary nickname.

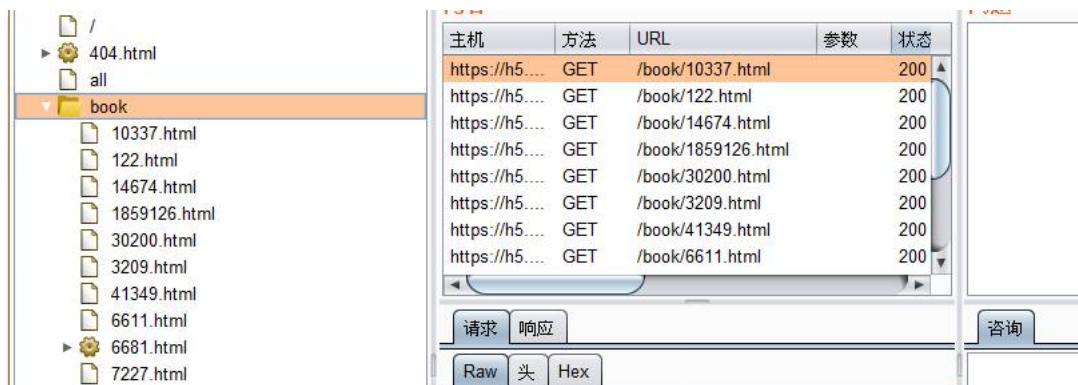
The code was successfully inserted into my profile bio: <https://user.xxx.com/h5/myUserInfo/>



On its own, that wasn't much use.

Next I wondered: could I find a place where someone could preview my profile just from my profile ID?

I used Burp Suite's crawler to enumerate the functional interfaces on the pages.



Eventually, in the book author pages at <https://h5.xxx.com/book/xxxx.html>, I found an interface for viewing an author's page.



目录 连载至401 更新于5个月前 >

3.0万 阅读过 0次 打赏 0张 推荐票 81.5万字 完本

水瑟嫣然 共2部作品 >

同类精选作品 换一换

少夫人她惊艳全球了
傅大总裁傅寒年被闯进公司的丑女人逼婚了。婚后，这女人上赶着要给他生娃娃。第一天，他说他对女人不感兴趣。第二天，她被无情踹下床。在失败的...
从一晚星河·总裁豪门

大总裁，小娇妻！
“女人，这半年你是我的。”一场意外，她和总裁有了数不清的纠缠。180天，她乖乖听话讨他欢心，步步为营生怕惹他不悦。终于，半年之期已到，...

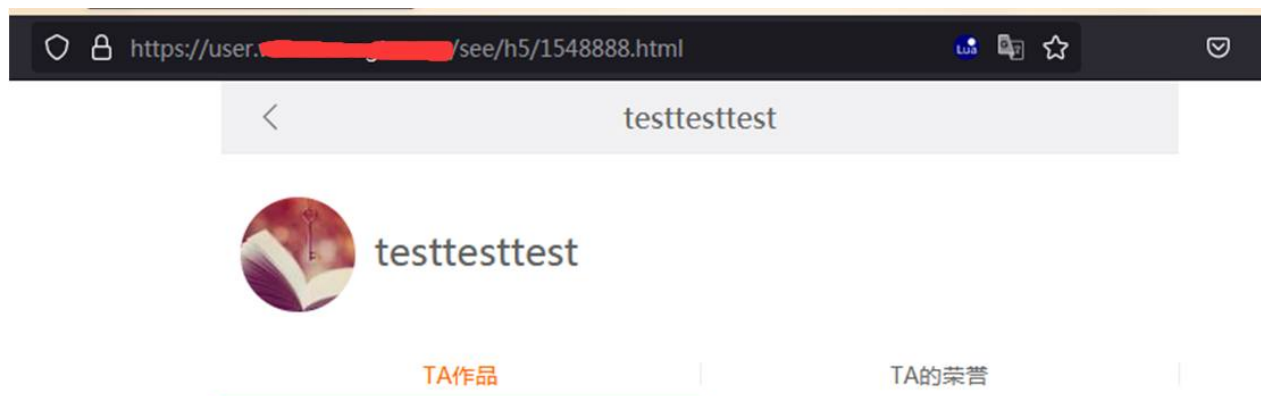
<https://user.xxx.com/see/h5/1574.html>

Try splicing in my own profile ID to test the waters:



<https://user.xxx.com/see/h5/1548888.html>

I thought a stored XSS would come easily — not so. After visiting the profile interface, the bio wasn't rendered into the page context at all.



It only displayed the author pen name I had used when logging in.



From the analysis above, all I needed now was one place to edit the author pen name with no input-length limit, and a stored XSS would be within reach.

After more interface crawling and directory scanning, I finally found a site with zero input filtering where I could change my author pen name: <https://user.xxx.com/www/>

The pen-name editing interface was well hidden inside the author profile section.

(Because several sites were linked together, I initially assumed this author profile edited the local account only — later I discovered this interface made the pen name take effect on the earlier system too.)



```
https://user.xxx.com/www/userinfo/contact.html
```

Just fill in the form as required — the key is to inject the payload into the author pen name.

Hit "save settings" and fire blindly!

基础资料 头像修改 **作者资料** 修改密码 账号绑定

tips: 成为作者后, 个人昵称会默认展示笔名

作者笔名: * name="penName" placeholder="作者笔名">

真实姓名:

身份证号:

邮寄地址:

联系电话:

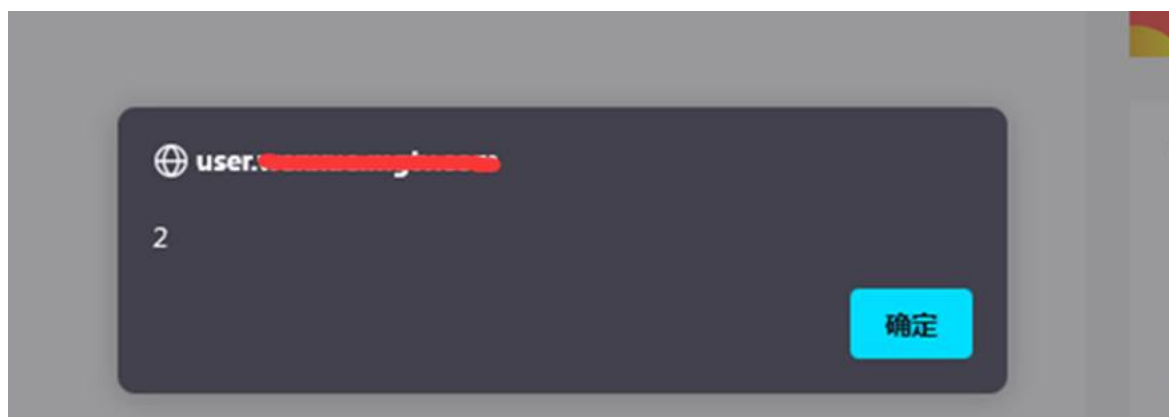
QQ 号码:

邮箱地址:

保存设置

And now, the moment of truth.

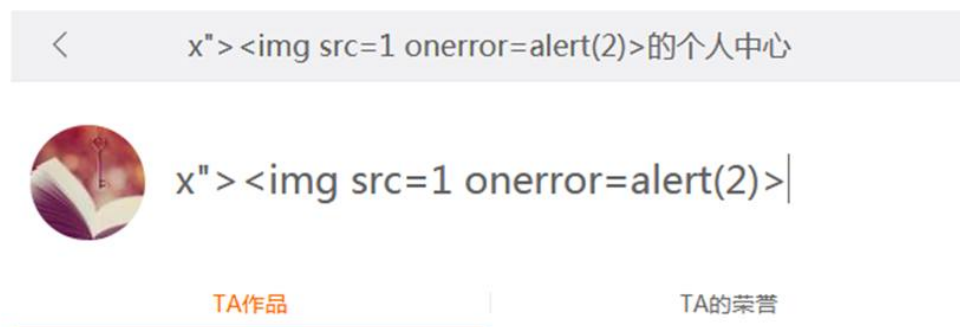
On that site itself, the bug triggers at: <https://user.xxx.com/www/xxx/?xx=1548888>



Then, back to the earlier step: visit the profile URL we spliced together.

Interface 1: <https://user.xxx.com/see/h5/1548888.html>

After visiting, the JavaScript was not executed in the context.



But the "TA's Honors" page made the bug fire successfully (that's interface 2).

Interface 2: <https://user.xxx.com/see/h5/xxx.html?id=1548888>

Visiting it triggered the vulnerability.

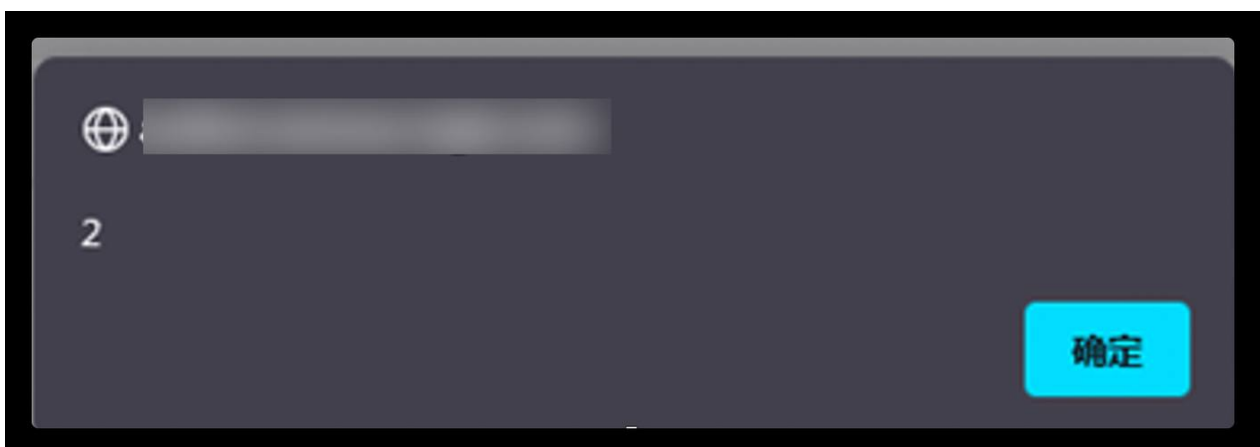


Whenever a user browses my honors page, this XSS fires — craft a malicious JS and you can steal their information.

Back to the registration system at the very beginning: it also contains an admin backend:

```
https://author.xxx.com/
```

The payload fired there too, in one of its contexts.



0x03: Summary

This bug was originally unexploitable, but careful digging — and one critical "edit" feature — turned it into exactly the result we wanted.

1. First-pass probing for classic vulnerabilities
2. Deep-diving into functional points
3. Clever use of Burp Suite's crawler to enumerate page URLs
4. Mapping out every location where the bug might trigger